TP5-2

Objectif :

- Utilisation des **migrations** pour la création et la gestion des structures de tables.
- Mise en place de **seeders et de factories** pour générer des données fictives dans la base de données.
- Utilisation du **Query Builder** pour interroger la base de données de manière fluide et performante.
- Implémentation de la **pagination automatique** et **manuelle** pour afficher efficacement les résultats sous forme de pages.

On souhaite gérer les examens régionaux à programmer au sein des établissements d'une région donnée. Pour cela, nous avons la table suivante :

Structure de la table examens :

Champ	Type	Contraintes
id	Clé primaire	
code_module	String (5)	Doit suivre le format "MXXX" (ex: M205)
titre_module	String (100)	
filiere	String (150)	Champ obligatoire
type	Enum	Valeurs possibles : théorique, pratique, synthèse
durée	Integer	Valeur positive, nullable, comprise entre 30 et 240, valeur par défaut 150
date_examen	Date	Doit être ultérieure à la date actuelle
created_at	Timestamp	
updated_at	Timestamp	

Travail demandé :

1. Création de la base de données et migration

- Créez une nouvelle application Laravel.
- Dans votre fichier de migration pour la table examens, ajoutez les contraintes mentionnées dans la structure ci-dessus.

2. Insertion de données avec un seeder

- Initialisez la table examens avec **au moins deux enregistrements** à l'aide d'un seeder.

3. Implémentation du CRUD (Create, Read, Update, Delete)

Développez les différentes fonctionnalités CRUD permettant de gérer les examens. Assurez-vous que :

- **La page "index" affiche la liste des examens** sous forme de tableau.
- **Les ID des examens sont des liens** menant à la page "show" qui affiche les détails de l'examen sélectionné.
- La modification et la suppression sont possibles.

4. Gestion des données avec les Factories

- Écrivez une factory pour la table examens permettant de générer des enregistrements réalistes.

5. Pagination des résultats

- Implémentez **une pagination automatique** avec Laravel pour l'affichage des examens sur la page "index".
- Ajoutez également **une pagination personnalisée** permettant à l'utilisateur de choisir le nombre d'éléments à afficher par page.

6. Manipulation des données avec Query Builder

- Écrivez une requête avec le **Query Builder** permettant de récupérer uniquement **les examens de type "théorique"**, triés par **date_examen décroissante**.
- Implémentez une **recherche par titre du module** permettant de filtrer les résultats affichés sur la page "index".